

Big Data and AI

In Renewable Energy Systems

Practical Use-Cases

Tsvetelina Kaneva, PhD

Department of Computer Systems and Technologies

University of Ruse, Bulgaria

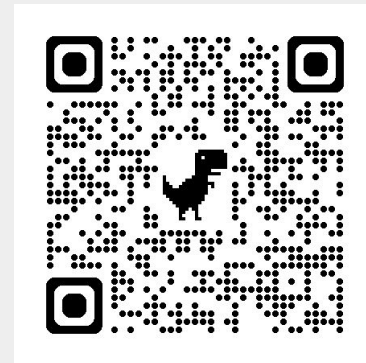
tskaneva@uni-ruse.bg

08.04.2026

Agenda

1. ML solutions to real-world problems
2. Team formation
3. Projects assignment
4. Google Colab and Jupyter Notebook 101
5. And other topics...

First things first...



**[github.com/tskaneva/
bip_2026_big_data_and_ai_in_res](https://github.com/tskaneva/bip_2026_big_data_and_ai_in_res)**

You can find the examples from this presentation in
the repository.



I. Case-Study: Solar Power Forecasting

Problem Definition

A utility company uses **big data + AI** to **predict solar energy generation** in order to optimize how electricity is distributed and stored.

- Solar power is intermittent
- Grid instability due to fluctuations
- Need for accurate forecasting and control

Solution

Machine learning gives the computer's the **ability to learn from data** rather than being explicitly programmed.

Therefore, it's an excellent choice to solve the problem in long-term.

How ML will help?

1. Forecast Solar Power - Predict PV output for the next hours/days

Example:

“Tomorrow at 2 PM → expected generation: 3.2 MW”

2. Optimize Energy Distribution

Decide when to:

- Store energy in batteries
- Send energy to the grid
- Reduce or shift loads

Typical ML flow

PROBLEM

Data
Collection

Data
Preparation

Model
Training

Model
Optimization

Model
Evaluation

Model
Deployment

What data could be used?

Huge volumes of data are continuously gathered from multiple sources:

- Weather data (*temperature, humidity, solar radiation, cloud cover*)
- Time data (*hour, day, season*)
- Historical PV generation data
- Energy consumption (*load profiles*)
- Satellite / sky images
- Grid status and storage levels

Data Sources

- **Open-source data** - multiple websites, the scientific community, etc.
- **Government data** - not always available, might be harder to obtain, could be regulated
- **Commercial data** - usually, paid and close-sourced
- **Personal data** - own sensors, own weather stations, harder to collect, can't be used right-away, if it's not been collected beforehand

```

    if (typeof function Array
ion F(e){var t=_[e]={};r
&e.stopOnFalse){r=!1;bre
:r&&(s=t,c(r))}return th
turn u=[],this},disable:
(){return p.fireWith(thi
state:function(){return
romise().done(n.resolve)
){n=s},t[1^e][2].disabl
(arguments),r=n.length,i
(r);r>t;t++)n[t]&&b.isFu
table><a href='/a'>a</a>
nput")[0],r.style.cssTex
ttribute("style")),hrefN

```

Let's code;

1. Import the libraries

In this first step, the Python libraries that will be used throughout the notebook are imported.

- **pandas** is used for loading and manipulating tabular data.
- **numpy** is useful for numerical operations.
- **matplotlib** is used for basic visualizations.

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import mean_absolute_error,
mean_squared_error, r2_score
```

2. Load the dataset

In this step, two separate datasets are loaded:

- one containing PV power measurements,
- and another containing weather data.

The **timestamp** column is converted to a **datetime** format to enable time-based operations such as *sorting, merging, and feature extraction*.

```
BASE_URL =  
"https://raw.githubusercontent.com/tskaneva/bip_2026_big_data_and_ai_in_res/main/data"  
  
# Load PV data  
pv_df = pd.read_csv(f"{BASE_URL}/sample_pv_data.csv")  
  
# Load weather data  
weather_df =  
pd.read_csv(f"{BASE_URL}/sample_weather_data.csv")  
  
# Convert timestamps  
pv_df['timestamp'] = pd.to_datetime(pv_df['timestamp'])  
weather_df['timestamp'] =  
pd.to_datetime(weather_df['timestamp'])  
  
pv_df.head()
```

2. Load the dataset

	timestamp	load	generated_power
0	2024-01-01 00:00:00	2.421	0.098
1	2024-01-01 01:00:00	2.490	0.000
2	2024-01-01 02:00:00	2.299	0.007
3	2024-01-01 03:00:00	2.128	0.000
4	2024-01-01 04:00:00	2.100	0.081

3. Merge the two datasets into one

In this step, the PV and weather datasets are merged into a single table using the timestamp column.

An **inner join** is used to retain **only the timestamps that exist in both datasets**.

This ensures that each row contains **complete information for both PV production and weather variables**.

The dataset is then **sorted chronologically**, which is **necessary for time series analysis**.

```
# Merge PV and weather data on timestamp
df = pd.merge(pv_df, weather_df, on='timestamp', how='inner')

# Sort chronologically
df = df.sort_values('timestamp').reset_index(drop=True)

print("Merged dataset shape:", df.shape)
display(df.head())
```

3. Merge the two datasets into one

Merged dataset shape: (2884, 7)

	timestamp	load	generated_power	temperature	humidity	wind_speed	solar_radiation
0	2024-01-01 00:00:00	2.421	0.098	3.160	70.852	2.358	586.69
1	2024-01-01 01:00:00	2.490	0.000	1.052	78.498	2.174	789.87
2	2024-01-01 02:00:00	2.299	0.007	1.432	82.185	5.112	248.06
3	2024-01-01 03:00:00	2.128	0.000	1.972	80.542	3.040	453.30
4	2024-01-01 04:00:00	2.100	0.081	-2.156	86.385	3.315	5.84

4. Inspect the dataset and check for missing values

In this step, the structure and quality of the dataset are examined.

The following aspects are checked:

- data types of each column,
- overall dataset structure,
- number of missing values per column.

Missing values are common in real-world datasets due to sensor errors or communication issues. Identifying them is an essential step before applying any machine learning model.

```
print("Dataset information:")
print(df.info())

print("Missing values per column:")
print(df.isnull().sum())
```

4. Inspect the dataset and check for missing values

```
Dataset information:
<class 'pandas.DataFrame'>
RangeIndex: 2884 entries, 0 to 2883
Data columns (total 7 columns):
#   Column                Non-Null Count  Dtype
---  -
0   timestamp             2884 non-null   datetime64[us]
1   load                  2884 non-null   float64
2   generated_power        2884 non-null   float64
3   temperature            2864 non-null   float64
4   humidity               2858 non-null   float64
5   wind_speed             2869 non-null   float64
6   solar_radiation        2843 non-null   float64
dtypes: datetime64[us](1), float64(6)
memory usage: 157.8 KB
None

Missing values per column:
timestamp      0
load           0
generated_power 0
temperature    20
humidity       26
wind_speed     15
solar_radiation 41
dtype: int64
```

5. Clean missing values

In this step, missing values are handled to ensure that the dataset is suitable for model training.

- A forward fill method is applied first, where missing values are replaced with the last available observation.
- A backward fill method is then applied to handle any remaining missing values at the beginning of the dataset.
- Finally, any remaining missing values are removed. This ensures that the dataset contains only complete records.

```
# Fill missing values using neighboring time steps
df = df.ffill()
df = df.bfill()

# Drop any remaining missing values
df = df.dropna()

print("Missing values after cleaning:")
print(df.isnull().sum())
```

```
Missing values after cleaning:
timestamp      0
load           0
generated_power 0
temperature    0
humidity        0
wind_speed     0
solar_radiation 0
dtype: int64
```

6. Create time-based features

In this step, new features are extracted from the timestamp. The following features are created:

- **hour** - represents the time of day and is strongly related to solar generation,
- **day_of_week** - may capture weekly patterns,
- **month** captures seasonal variations.

These features help the model learn patterns related to time, which are important in renewable energy forecasting.

```
df['hour'] = df['timestamp'].dt.hour
df['day_of_week'] = df['timestamp'].dt.dayofweek
df['month'] = df['timestamp'].dt.month

display(df[['timestamp', 'hour', 'day_of_week', 'month']].head())
```

	timestamp	hour	day_of_week	month
0	2024-01-01 00:00:00	0	0	1
1	2024-01-01 01:00:00	1	0	1
2	2024-01-01 02:00:00	2	0	1
3	2024-01-01 03:00:00	3	0	1
4	2024-01-01 04:00:00	4	0	1

7. Create the target for hour-ahead prediction

In this step, the prediction target is defined for an hour-ahead forecasting task. The goal is to predict PV power at the next time step (one hour ahead) using current information. This is achieved by **shifting the PV power column by one step forward**. As a result:

- the current row contains input features,
- the target corresponds to the next hour. The last row is removed because it does not have a future value available.

```
# Create the target variable: PV power one hour ahead
df['target_power'] = df['generated_power'].shift(-1)

# Remove the last row because it has no future target
df = df.dropna().reset_index(drop=True)

display(df[['timestamp', 'generated_power',
            'target_power']].head())
```

	timestamp	generated_power	target_power
0	2024-01-01 00:00:00	0.098	0.000
1	2024-01-01 01:00:00	0.000	0.007
2	2024-01-01 02:00:00	0.007	0.000
3	2024-01-01 03:00:00	0.000	0.081
4	2024-01-01 04:00:00	0.081	0.000

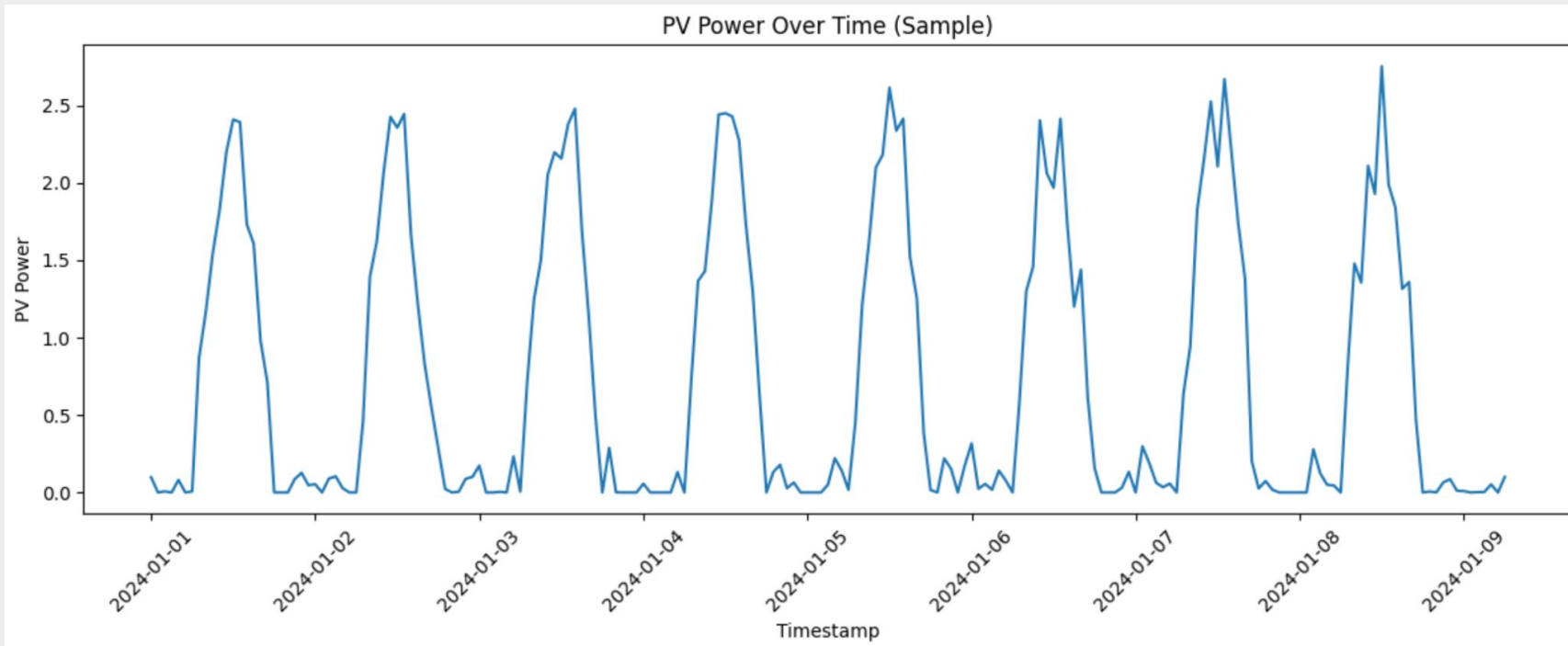
8. Explore the data visually

In this step, a portion of the PV power data is visualized over time. This visualization helps to observe:

- daily generation patterns,
- periods of zero production (e.g., nighttime),
- variability due to weather conditions. Visual inspection of the data is useful for understanding underlying patterns before applying machine learning models.

```
plt.figure(figsize=(12, 5))
plt.plot(df['timestamp'][:200], df['generated_power'][:200])
plt.title("PV Power Over Time (Sample)")
plt.xlabel("Timestamp")
plt.ylabel("PV Power")
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```

8. Explore the data visually



N.B. This dataset is synthetic, and too perfect.

9. Select the input features and target

In this step, the input features (X) and the target variable (y) are defined. The selected features include:

- current PV power,
- solar radiance,
- temperature,
- humidity,
- time-based features such as hour and month.

The target variable is the PV power at the next time step, representing the hour-ahead forecast.

```
features = ['generated_power', 'solar_radiation',  
            'temperature', 'humidity', 'hour', 'month']  
  
X = df[features]  
y = df['target_power']  
  
print("Selected features:")  
print(features)
```

```
Selected features:  
['generated_power', 'solar_radiation', 'temperature', 'humidity', 'hour', 'month']
```


10. Split the data into training and testing sets

In this step, the dataset is divided into **training** and **testing** subsets. A **chronological split** is used instead of a random split. This is important in time series problems because **future data must not be used to predict past values**.

Typically:

- the first 80% of the data is used for training,
- the remaining 20% is used for testing. This approach better reflects real-world forecasting scenarios.

```
X_train, X_test, y_train, y_test = train_test_split(  
    X, y, test_size=0.2, random_state=42  
)  
  
print('Training samples:', len(X_train))  
print('Testing samples:', len(X_test))
```

```
Training samples: 2306  
Testing samples: 577
```

11. Train the Random Forest model and testing sets

The Random Forest Regressor is well-suited for tabular data and can model nonlinear relationships between variables. The following parameters are used:

- `n_estimators=100` - defines the number of trees in the forest. A higher number generally improves performance but increases computational cost.
- `max_depth=10` - limits the depth of each tree, helping to prevent overfitting.
- `random_state=42` - ensures reproducibility of results.

```
model = RandomForestRegressor(  
    n_estimators=100,  
    max_depth=10,  
    random_state=42  
)  
  
model.fit(X_train, y_train)
```



RandomForestRegressor



RandomForestRegressor(max_depth=10, random_state=42)

12. Make predictions on the test set

In this step, the trained model is used to generate predictions on the test dataset.

These predictions represent the estimated PV power one hour ahead and will be compared with the actual values to evaluate model performance.

```
y_pred = model.predict(X_test)
print("First 10 predictions:")
print(y_pred[:10])
```

First 10 predictions:

```
[1.6356664  2.70649093 0.05870674 0.69644024 2.26409757 0.74539681
 0.08164731 0.0382905  1.88895701 0.07552897]
```

13. Evaluate the model performance

In this step, the performance of the model is evaluated using three common regression metrics.

These metrics provide insight into how accurately the model predicts PV power.

```
mae = mean_absolute_error(y_test, y_pred)
rmse = np.sqrt(mean_squared_error(y_test, y_pred))
r2 = r2_score(y_test, y_pred)

print(f'MAE = {mae:.4f}')
print(f'RMSE = {rmse:.4f}')
print(f'R2 = {r2:.4f}')
```

MAE = 0.1326
RMSE = 0.2071
R² = 0.9659

14. Visualize actual vs predicted values

In this step, the predicted values are compared with the actual values using a plot.

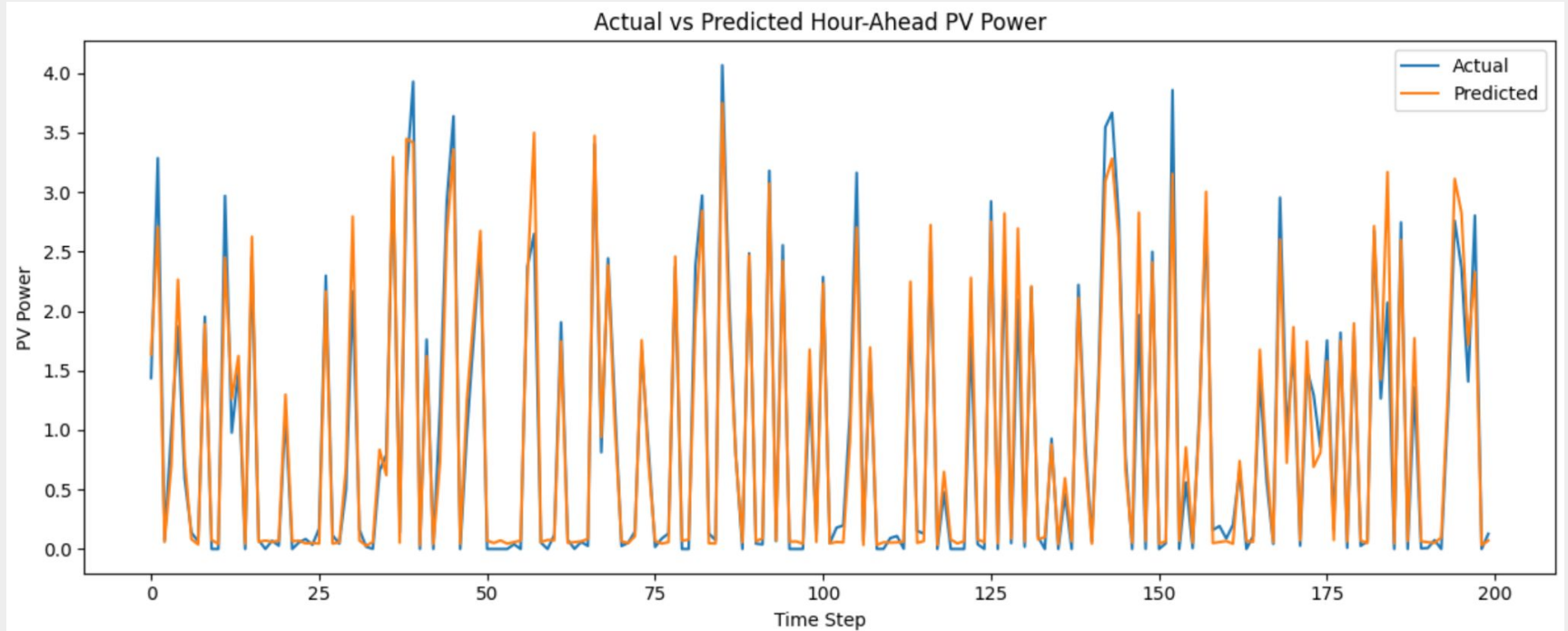
This visualization helps to assess:

- how well the model follows the overall trend,
- how accurately peaks and drops are captured,
- where significant prediction errors occur.

Plotting a subset of the data improves readability.

```
plt.figure(figsize=(12, 5))
plt.plot(y_test.values[:200], label='Actual')
plt.plot(y_pred[:200], label='Predicted')
plt.title("Actual vs Predicted Hour-Ahead PV Power")
plt.xlabel("Time Step")
plt.ylabel("PV Power")
plt.legend()
plt.tight_layout()
plt.show()
```

14. Visualize actual vs predicted values



15. Analyze feature importance

In this step, the importance of each input feature is analyzed.

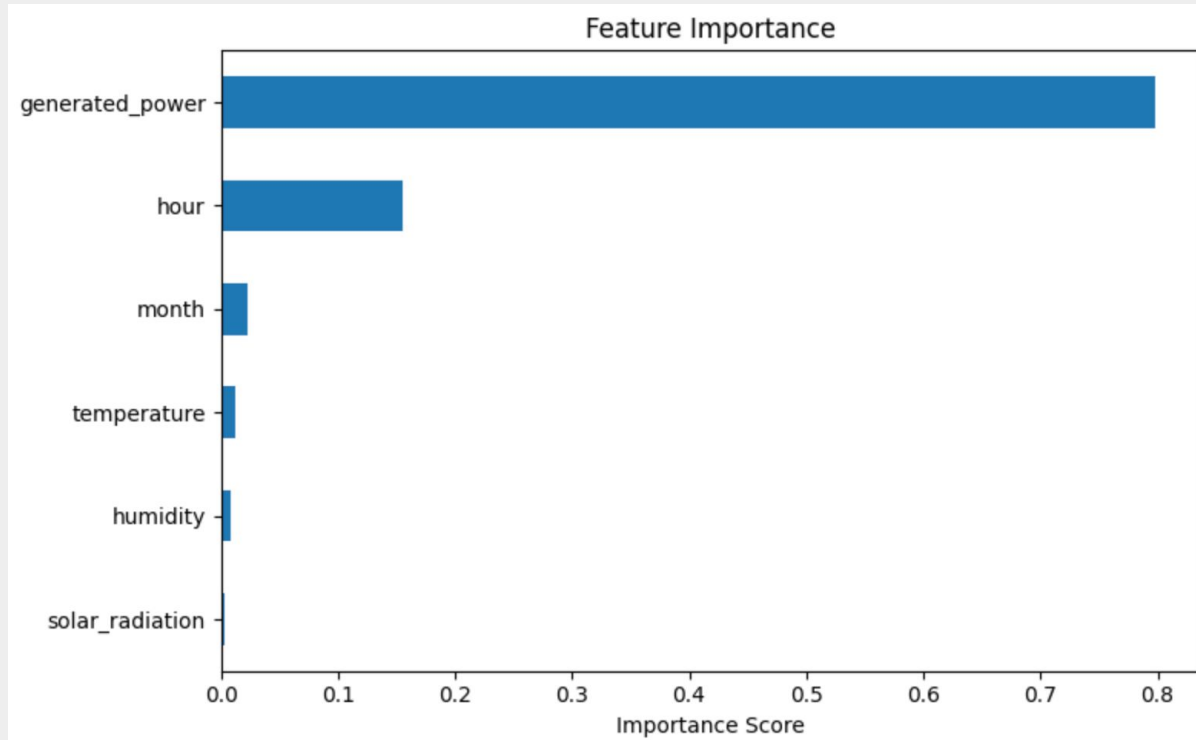
The Random Forest model provides an estimate of how much each feature contributes to the prediction.

This analysis helps to:

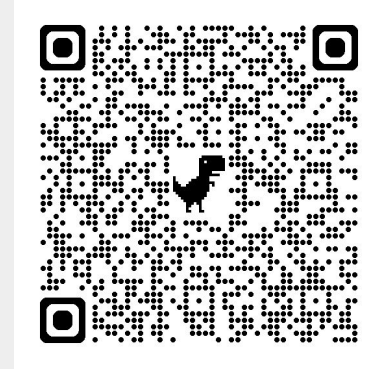
- identify the most influential variables,
- understand the relationship between inputs and PV power,
- validate domain knowledge (e.g., the importance of irradiance).

```
feature_importance = pd.Series(model.feature_importances_,  
                                index=features)  
feature_importance =  
feature_importance.sort_values(ascending=False)  
  
plt.figure(figsize=(8, 5))  
feature_importance.plot(kind='barh')  
plt.title("Feature Importance")  
plt.xlabel("Importance Score")  
plt.tight_layout()  
plt.show()
```

15. Analyze feature importance



Full Jupyter Notebook here:



[https://github.com/tskaneva/bip_2026_big_data_and_ai_in_res/blob/main/workshop%20notebooks/01 solar power forecasting.ipynb](https://github.com/tskaneva/bip_2026_big_data_and_ai_in_res/blob/main/workshop%20notebooks/01_solar_power_forecasting.ipynb)

**Let's try something harder and more
believable**

The background of the slide features a dark blue, semi-transparent overlay of financial data. On the left, there are two candlestick charts showing price movements. On the right, a list of stock indices is visible, including SPX, NDQ, DJI, VIX, and DXY, each accompanied by a small circular icon. A prominent price tag for 138.15 is shown in a red box, with a timestamp of 02:53 below it. The overall aesthetic is professional and data-driven.

II. Case-Study: Energy Price Prediction for Bulgaria

Problem?

Do you know when is **cheapest** to start the washing machine?

You are on the “*free*” market, where electricity consumers can choose their suppliers and negotiate contracts, rather than being tied to a single local utility. Contracts often include fixed or **variable rates** tailored to consumption, reducing overall costs. A free energy market with variable pricing **allows electricity costs to fluctuate based on real-time market conditions**, such as supply, demand, and fuel prices.

Solution

in 31 steps.
(more or less...)

The project demonstrates a full market price forecasting pipeline:

- loading and cleaning multiple datasets
- merging load with price data
- creating time, lag, and rolling features
- building a baseline forecast
- training a Gradient Boosting model
- evaluating and visualizing results

1. Load the libraries

Basically, the same as the first example.

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

from sklearn.ensemble import GradientBoostingRegressor
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
```

2. Load the datasets (but first find them...)

In many cases, you'll need datasets from *multiple sources and formats*. You might have separate datasets for each feature, or time period, etc.

Some of your files might be in XLS, others in CSV, or in a database...

Your very first step is to decide *what data you'll need*.

After that, the search begins.

In this example the Transparency Platform for Electricity generation, transportation and consumption for the European market will be used.

2. Load the datasets (but first find them..)

entsoe Transparency platform
Electricity generation, transportation and consumption for the European market.

01/04/2026 18:54 CET/CEST

Sign in

Bidding Zones

BZN|BG

Close

Market Load Generation Transmission Outages Balancing Operations

Energy Prices

01/01/2025

SELECT ITEM(S)

Select all Unselect all

WITHOUT SEQUENCE - DAY-AHEAD

Day-ahead

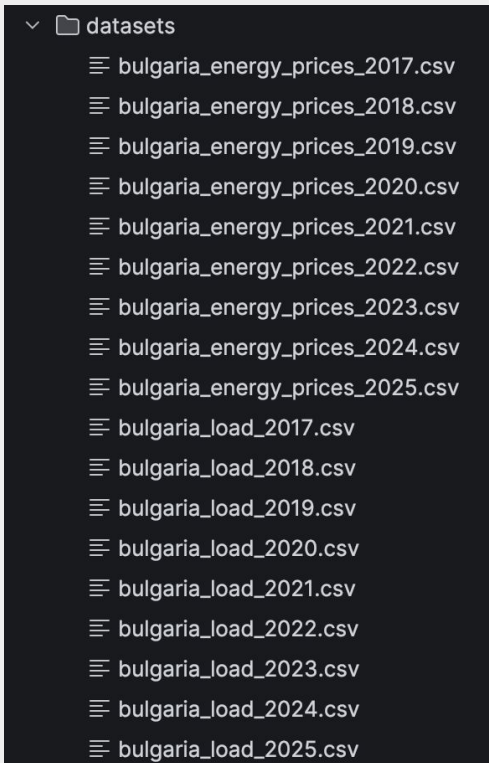
MTU	Without Sequence
	Day-ahead (EUR/MWh)
01/01/2025 00:00 - 01/01/2025 01:00	138.70
01/01/2025 01:00 - 01/01/2025 02:00	134.06
01/01/2025 02:00 - 01/01/2025 03:00	124.42
01/01/2025 03:00 - 01/01/2025 04:00	84.48
01/01/2025 04:00 - 01/01/2025 05:00	64.28

2. Load the datasets (but first find them...)

A minor inconvenience...

In many, real-life scenarios, the data comes in multiple files.

Or in this case - **18 files** that should be processed.



2. Load the datasets (but first find them..)

```
"MTU (CET/CEST)","Area","Sequence","Day-ahead Price (BGN/MWh)","Intraday Period (CET/CEST)","Intraday Price (BGN/MWh)"
"01/01/2017 00:00:00 - 01/01/2017 01:00:00","BZN|BG","Without Sequence","44.01","",""
"01/01/2017 01:00:00 - 01/01/2017 02:00:00","BZN|BG","Without Sequence","38.08","",""
"01/01/2017 02:00:00 - 01/01/2017 03:00:00","BZN|BG","Without Sequence","36.75","",""
"01/01/2017 03:00:00 - 01/01/2017 04:00:00","BZN|BG","Without Sequence","36.91","",""
"01/01/2017 04:00:00 - 01/01/2017 05:00:00","BZN|BG","Without Sequence","39.08","",""
"01/01/2017 05:00:00 - 01/01/2017 06:00:00","BZN|BG","Without Sequence","39.49","",""
"01/01/2017 06:00:00 - 01/01/2017 07:00:00","BZN|BG","Without Sequence","39.17","",""
```

Price dataset

```
"MTU (CET/CEST)","Area","Sequence","Day-ahead Price (EUR/MWh)","Intraday Period (CET/CEST)","Intraday Price (EUR/MWh)"
"01/01/2024 00:00:00 - 01/01/2024 01:00:00","BZN|BG","Without Sequence","0.10","",""
"01/01/2024 01:00:00 - 01/01/2024 02:00:00","BZN|BG","Without Sequence","0.01","",""
"01/01/2024 02:00:00 - 01/01/2024 03:00:00","BZN|BG","Without Sequence","0.04","",""
"01/01/2024 03:00:00 - 01/01/2024 04:00:00","BZN|BG","Without Sequence","0.01","",""
"01/01/2024 04:00:00 - 01/01/2024 05:00:00","BZN|BG","Without Sequence","0.01","",""
"01/01/2024 05:00:00 - 01/01/2024 06:00:00","BZN|BG","Without Sequence","0.01","",""
"01/01/2024 06:00:00 - 01/01/2024 07:00:00","BZN|BG","Without Sequence","0.04","",""
"01/01/2024 07:00:00 - 01/01/2024 08:00:00","BZN|BG","Without Sequence","0.01","",""
"01/01/2024 08:00:00 - 01/01/2024 09:00:00","BZN|BG","Without Sequence","0.01","",""
"01/01/2024 09:00:00 - 01/01/2024 10:00:00","BZN|BG","Without Sequence","0.04","",""
"01/01/2024 10:00:00 - 01/01/2024 11:00:00","BZN|BG","Without Sequence","0.06","",""
"01/01/2024 11:00:00 - 01/01/2024 12:00:00","BZN|BG","Without Sequence","0.54","",""
```

2. Load the datasets (but first find them...)

Load
dataset

```
"MTU (CET/CEST)","Area","Actual Total Load (MW)","Day-ahead Total Load Forecast (MW)"
"01/01/2022 00:00 - 01/01/2022 01:00","BZN|BG","4060.00","4145.00"
"01/01/2022 01:00 - 01/01/2022 02:00","BZN|BG","3888.00","4031.00"
"01/01/2022 02:00 - 01/01/2022 03:00","BZN|BG","3751.00","3984.00"
"01/01/2022 03:00 - 01/01/2022 04:00","BZN|BG","3668.00","3980.00"
"01/01/2022 04:00 - 01/01/2022 05:00","BZN|BG","3616.00","4013.00"
"01/01/2022 05:00 - 01/01/2022 06:00","BZN|BG","3612.00","4126.00"
"01/01/2022 06:00 - 01/01/2022 07:00","BZN|BG","3656.00","4316.00"
"01/01/2022 07:00 - 01/01/2022 08:00","BZN|BG","3833.00","4176.00"
"01/01/2022 08:00 - 01/01/2022 09:00","BZN|BG","4083.00","4816.00"
"01/01/2022 09:00 - 01/01/2022 10:00","BZN|BG","4253.00","4828.00"
```

Helper functions for data loading

```
def open_price_datasets(year: int):
    dataset_path = f"../datasets/bulgaria_energy_prices_{year}.csv"
    tmp_df = pd.read_csv(dataset_path)
    BGN_PER_EUR = 1.95583
    timestamp_col = 'MTU (CET/CEST)'
    eur_col = 'Day-ahead Price (EUR/MWh)'
    bgn_col = 'Day-ahead Price (BGN/MWh)'

    if eur_col in tmp_df.columns:
        price_df = tmp_df[[timestamp_col, eur_col]].rename(columns={
            eur_col: 'price',
            timestamp_col: 'timestamp'
        })
    elif bgn_col in tmp_df.columns:
        price_df = tmp_df[[timestamp_col, bgn_col]].rename(columns={
            bgn_col: 'price',
            timestamp_col: 'timestamp'
        })
    price_df['price'] = price_df['price'] / BGN_PER_EUR
    else:
        raise KeyError(f"Neither '{eur_col}' nor '{bgn_col}' was found in {dataset_path}")
    return price_df
```

Helper functions for data loading

```
def open_load_datasets(year: int):  
    dataset_path = f"../datasets/bulgaria_load_{year}.csv"  
    tmp_df = pd.read_csv(dataset_path)  
  
    return tmp_df[['MTU (CET/CEST)', 'Actual Total Load (MW)']].rename(columns={  
        'Actual Total Load (MW)': 'load',  
        'MTU (CET/CEST)': 'timestamp'  
    })
```

3. Load the datasets

```
years = range(2017, 2026)
load_df = pd.DataFrame()
price_df = pd.DataFrame()

for year in years:
    price_df = pd.concat([price_df, open_price_datasets(year)], ignore_index=True)
    load_df = pd.concat([load_df, open_load_datasets(year)], ignore_index=True)
```

	timestamp	price
0	01/01/2017 00:00:00 - 01/01/2017 01:00:00	22.501956
1	01/01/2017 01:00:00 - 01/01/2017 02:00:00	19.469995
2	01/01/2017 02:00:00 - 01/01/2017 03:00:00	18.789977
3	01/01/2017 03:00:00 - 01/01/2017 04:00:00	18.871783
4	01/01/2017 04:00:00 - 01/01/2017 05:00:00	19.981287

	timestamp	load
0	01/01/2017 00:00 - 01/01/2017 01:00	4803.0
1	01/01/2017 01:00 - 01/01/2017 02:00	4579.0
2	01/01/2017 02:00 - 01/01/2017 03:00	4423.0
3	01/01/2017 03:00 - 01/01/2017 04:00	4331.0
4	01/01/2017 04:00 - 01/01/2017 05:00	4286.0

4. Create a helper function for timestamp handling

```
def timestamp_handle(df):  
    ts = df['timestamp'].str.partition(' - ')[0]  
    ts = ts.str.replace(r' \((?:CET|CEST)\)', '', regex=True)  
    ts = pd.to_datetime(ts, dayfirst=True)  
    df = df.copy()  
    df['timestamp'] = ts  
  
    return df.set_index('timestamp').sort_index().resample('h').mean()
```

The timestamps in the source files contain time intervals and timezone labels such as CET or CEST. The function:

- keeps only the starting part of the interval
- removes the timezone text
- converts the values to Python datetime objects
- sets the timestamp as index
- sorts the records
- resamples the data to hourly resolution

4. ... 7. Handle missing values

```
df['price'] = df['price'].interpolate(method='time').ffill().bfill()  
df['load'] = df['load'].interpolate(method='time').ffill().bfill()
```

The missing values in the `price` and `load` columns are filled using:

- time-based interpolation
- forward fill
- backward fill

This ensures the model receives a continuous hourly time series without gaps.

8.... 10. Create time-based features

```
df["hour"] = df.index.hour
df["day_of_week"] = df.index.dayofweek
df["month"] = df.index.month
df["is_weekend"] = (df["day_of_week"] ≥ 5).astype(int)
df["day_of_month"] = df.index.day
df["day_of_year"] = df.index.dayofyear
df["week_of_year"] = df.index.isocalendar().week.astype(int)
df["quarter"] = df.index.quarter
df["is_morning_peak"] = df["hour"].isin([7, 8, 9]).astype(int)
df["is_evening_peak"] = df["hour"].isin([18, 19, 20, 21]).astype(int)
```

12. Create cyclical features

```
df["hour_sin"] = np.sin(2 * np.pi * df["hour"] / 24)
df["hour_cos"] = np.cos(2 * np.pi * df["hour"] / 24)

df["dow_sin"] = np.sin(2 * np.pi * df["day_of_week"] / 7)
df["dow_cos"] = np.cos(2 * np.pi * df["day_of_week"] / 7)

df["month_sin"] = np.sin(2 * np.pi * df["month"] / 12)
df["month_cos"] = np.cos(2 * np.pi * df["month"] / 12)
```

In this step, the hour and day-of-week variables are transformed into sine and cosine representations.

This is useful because these variables are cyclical.

For example, *hour 23 and hour 0 are close in time, but their raw numeric values are far apart.*

13. Create lagging features

```
df["price_lag_1"] = df["price"].shift(1)
df["price_lag_2"] = df["price"].shift(2)
df["price_lag_3"] = df["price"].shift(3)
df["price_lag_24"] = df["price"].shift(24)
df["price_lag_48"] = df["price"].shift(48)
df["price_lag_168"] = df["price"].shift(168)

df["load_lag_1"] = df["load"].shift(1)
df["load_lag_24"] = df["load"].shift(24)
df["load_lag_48"] = df["load"].shift(48)
df["load_lag_168"] = df["load"].shift(168)
```

Lag features provide the model with information about recent history, which is especially important in time series forecasting.

- the previous 1, 2, 3 hours
- the previous 24 and 48 hours
- the previous week (168 hours)

These predictors often carry strong forecasting power because electricity prices and load are highly dependent on recent past values.

14. Create rolling features

```
df["price_roll_mean_6"] = df["price"].shift(1).rolling(window=6).mean()
df["price_roll_mean_12"] = df["price"].shift(1).rolling(window=12).mean()
df["price_roll_mean_24"] = df["price"].shift(1).rolling(window=24).mean()
df["price_roll_std_24"] = df["price"].shift(1).rolling(window=24).std()
df["price_roll_max_24"] = df["price"].shift(1).rolling(window=24).max()
df["price_roll_min_24"] = df["price"].shift(1).rolling(window=24).min()

df["load_roll_mean_6"] = df["load"].shift(1).rolling(window=6).mean()
df["load_roll_mean_24"] = df["load"].shift(1).rolling(window=24).mean()
df["load_roll_std_24"] = df["load"].shift(1).rolling(window=24).std()
```

Rolling features summarize recent behavior over short windows, such as:

- rolling means
- rolling standard deviation
- rolling maximum
- rolling minimum

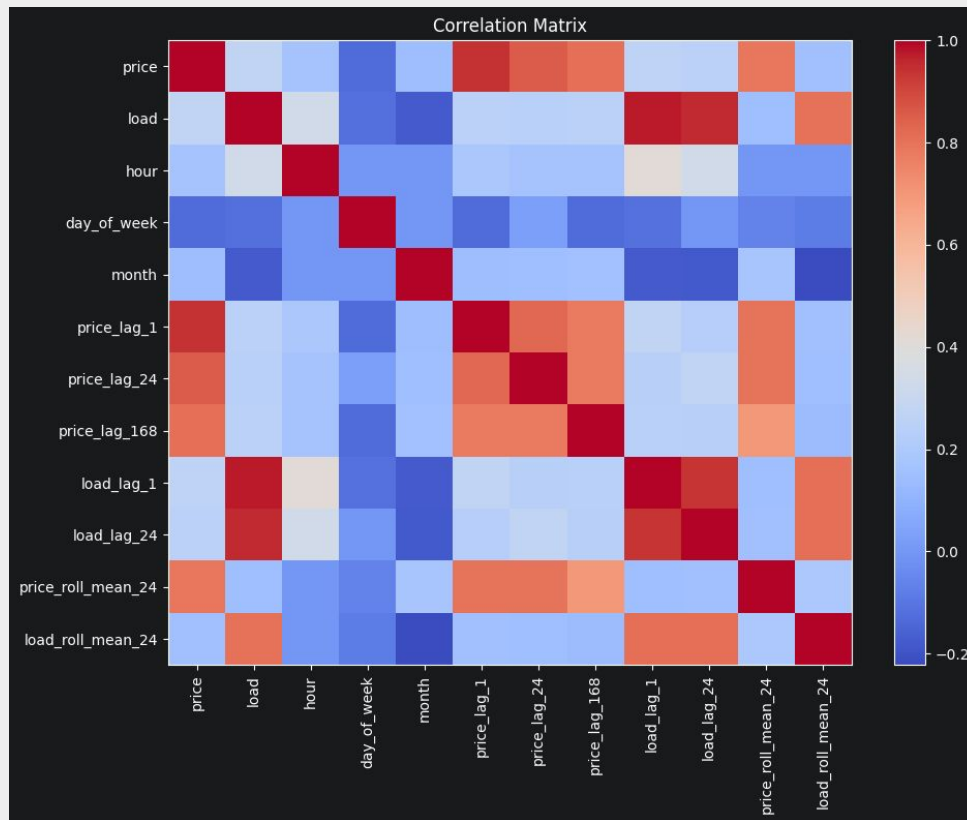
These features help the model capture short-term trends and local volatility in price and load.

15. Correlation Analysis

This provides a quick overview of the linear relationships between:

- the target-related variables
- the load values
- the time-based features
- the lagged and rolling predictors

This step is useful for understanding which features may carry useful predictive information.



16...18. Define the features and the target variable

```
feature_columns = [  
    "Load", "hour", "day_of_week", "month", "is_weekend", "day_of_month", "day_of_year", "week_of_year",  
    "quarter", "is_morning_peak", "is_evening_peak", "hour_sin", "hour_cos", "dow_sin", "dow_cos",  
    "month_sin", "month_cos", "price_lag_1", "price_lag_2", "price_lag_3", "price_lag_24", "price_lag_48",  
    "price_lag_168", "load_lag_1", "load_lag_24", "load_lag_48", "load_lag_168", "price_roll_mean_6",  
    "price_roll_mean_12", "price_roll_mean_24", "price_roll_std_24", "price_roll_max_24",  
    "price_roll_min_24", "load_roll_mean_6", "load_roll_mean_24", "load_roll_std_24"  
]  
  
X = df[feature_columns]  
y = df["target_price_next_hour"]
```

Started with **three** columns - *timestamp*, *load*, and *price*.
Ended up with **36** (!) useful features.

19. Split the data chronologically

```
n = len(df)

train_end = int(n * 0.70)
val_end = int(n * 0.85)

X_train = X.iloc[:train_end]
y_train = y.iloc[:train_end]

X_val = X.iloc[train_end:val_end]
y_val = y.iloc[train_end:val_end]

X_test = X.iloc[val_end:]
y_test = y.iloc[val_end:]
```

A chronological split is used instead of a random split because time series forecasting must preserve temporal order.

This avoids information leakage from the future into the past.

20. Create a baseline model

```
baseline_val_pred = X_val["price_lag_1"]  
baseline_test_pred = X_test["price_lag_1"]
```

In this step, a simple baseline forecast is created using the previous hour price (price_lag_1).

This baseline is important because it provides a reference point. A machine learning model should perform better than this simple persistence-based approach to justify its use.

21. Evaluate the baseline model

The model shows:

- Moderate error (MAE ~26–35)
- Quite high RMSE (~46–60) → large mistakes exist
- Low R^2 (~0.30–0.35) → weak explanatory power

The model is learning something, but not enough to be reliable.

N.B.: The purpose of this presentation is to show you the steps and the source code for the main, basic ML tasks. We're not trying to build a good model. That will be your task later.

Baseline Validation Results

MAE : 26.2905

RMSE : 46.3927

R^2 : 0.3517

Baseline Test Results

MAE : 35.1406

RMSE : 60.2906

R^2 : 0.3006

22. Train a Gradient Boosting Model

```
model = GradientBoostingRegressor(  
    n_estimators=200,  
    learning_rate=0.05,  
    max_depth=5,  
    random_state=42  
)  
  
model.fit(X_train, y_train)
```

The Gradient Boosting regression model is created and trained on the training dataset.

The selected model parameters control:

- the number of boosting stages
- the learning rate
- the depth of each individual tree

23. Make validation and test predictions

```
val_pred = model.predict(X_val)  
test_pred = model.predict(X_test)
```

The validation predictions are used to inspect model behavior before final testing, while the test predictions are reserved for final evaluation.

24. Evaluate the model

```
evaluate_model(y_val, val_pred, "Gradient Boosting Validation Results")  
evaluate_model(y_test, test_pred, "Gradient Boosting Test Results")
```

Gradient Boosting Validation Results

MAE : 18.2121

RMSE : 32.7138

R^2 : 0.6777

Gradient Boosting Test Results

MAE : 23.8023

RMSE : 41.9327

R^2 : 0.6617

Compared to baseline:

- MAE ↓ significantly ($\approx 35 \rightarrow \approx 24$ on test)
- RMSE ↓ a lot ($\approx 60 \rightarrow \approx 42$)
- R^2 ↑ massively ($\approx 0.30 \rightarrow \approx 0.66$)

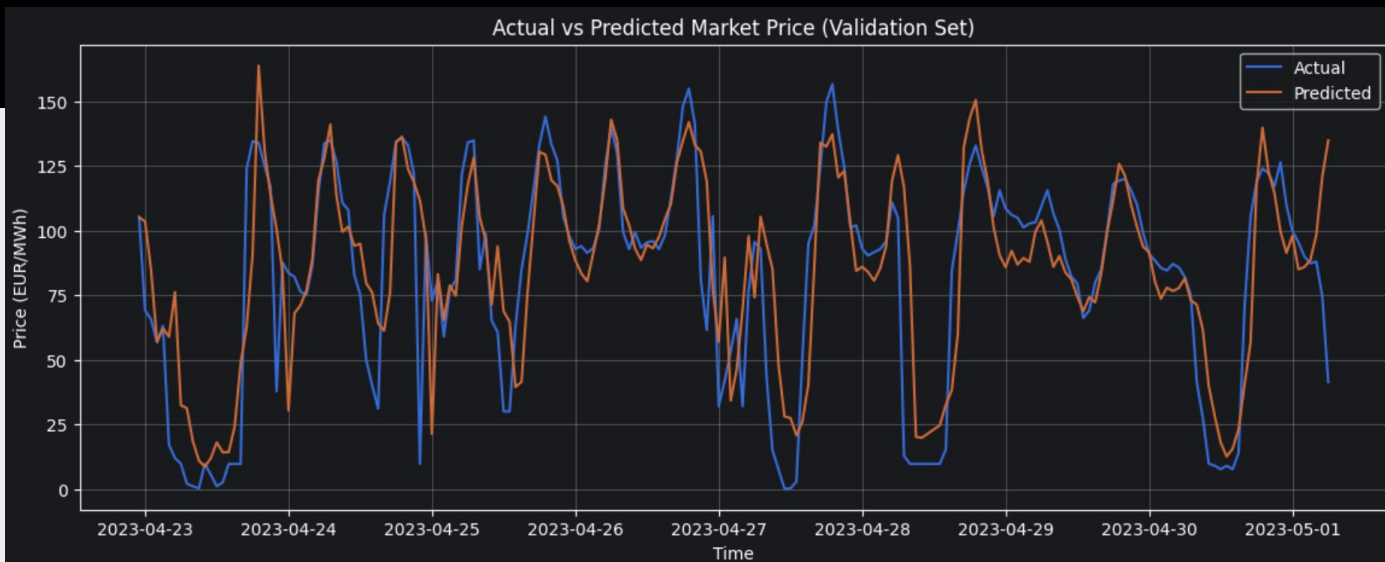
The model now explains ~66–68% of the variance.

Validation vs Test (Generalization Check):

- Very small drop in R^2 (~ 0.016) → excellent
- Errors increase slightly → normal
- Good generalization
- No serious overfitting

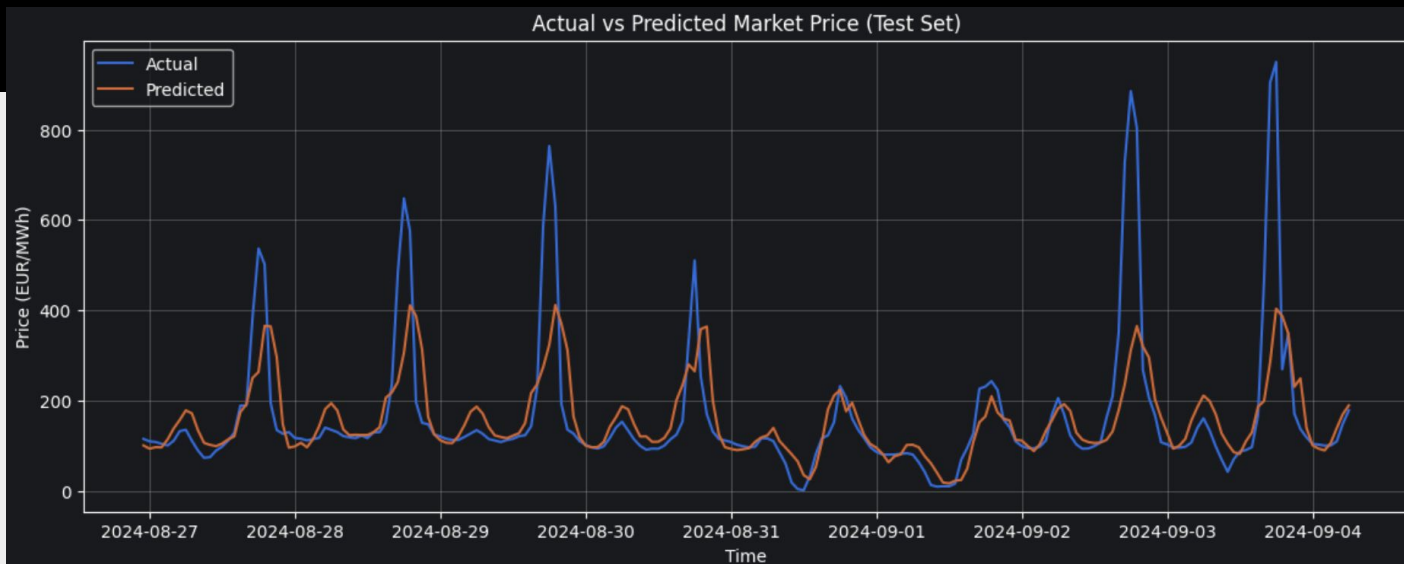
25. A validation plot

```
plt.figure(figsize=(14, 5))
plt.plot(y_val.index[:200], y_val.iloc[:200], label="Actual")
plt.plot(y_val.index[:200], val_pred[:200], label="Predicted")
plt.title("Actual vs Predicted Market Price (Validation Set)")
plt.xlabel("Time")
plt.ylabel("Price (EUR/MWh)")
plt.legend()
plt.grid(True)
plt.show()
```



26. A plot for the actual vs. predicted values on the test

```
plt.figure(figsize=(14, 5))
plt.plot(y_test.index[:200], y_test.iloc[:200], label="Actual")
plt.plot(y_test.index[:200], test_pred[:200], label="Predicted")
plt.title("Actual vs Predicted Market Price (Test Set)")
plt.xlabel("Time")
plt.ylabel("Price (EUR/MWh)")
plt.legend()
plt.grid(True)
plt.show()
```



28...33 Some steps for analysis

28. Add error analysis - to inspect the quality of the model and summarize the model's mistakes

29. Plot the prediction errors - the time-series error plot shows when large mistakes occur, while the histogram shows how the errors are distributed overall.

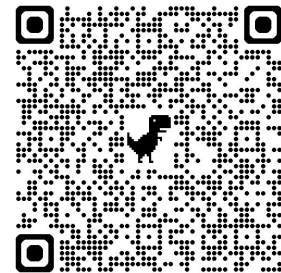
30. Performance by hour of day - helps identify whether the model performs better during some parts of the day and worse during others, such as peak demand periods.

31. Directional Accuracy - useful in market forecasting because sometimes predicting the direction correctly is just as important as predicting the exact value.

32. Feature importance - helps show which input variables contributed most to the forecasting task.

33. Plot the feature importance - provides a clear visual summary of which predictors had the strongest effect on the model's decisions.

34.+ What are your next steps



All of the analysis steps are done in order to see the models' performance, strengths and lows.

Your next steps are for **model optimization** - choose other features, make new features, remove some of the features, find additional data, find more data, tweak the model's parameters, choose another model, compare models...

Full notebook for this example -

https://github.com/tskaneva/bip_2026_big_data_and_ai_in_res/blob/main/workshop%20notebooks/04_real_data_electricity_price_forecasting_notebook.ipynb

Questions?

We will continue after the coffee break.

Teams.

As International as we can.

1

Team 1

- Croatia — Ana Curic
- Romania — Teodor-Stefan Zaharia
- Poland — Piotr Perkowski
- Germany — Harsh Doshi
- France — Axel Romani

2

Team 2

- Croatia — Ivona Rados
- Romania — Dragos-Constantin Toma
- Poland — Maciej Wozniak
- Germany — Maria Matyukhina
- Poland — Kacper Korol

3

Team 3

- Croatia — Dominik Benak
- Romania — Stelian Alexandru
- Poland — Anna Nalewajko
- Germany — David Hock
- Poland — Hanna Barylak

4

Team 4

- Croatia — Mihael Mežnarić
- Romania — Robert Stefan Cotet
- Poland — Jan Chalupka
- Germany — Lien Bradley
- Poland — Julia Bieganska

5

Team 5

- Croatia — Nikolina Pejic
- Romania — Alexandru-Nicolae Badinga
- Poland — Julia Ordecka
- Bulgaria — Simona Dragieva
- France — Flavien Norie

6

Team 6

- Croatia — Filip Bicanic
- Romania — Danu Cislaru
- Poland — Dominika Filipiak
- Bulgaria — Emilia Velikova
- Poland — Kacper Grobelny

7

Team 7

- Croatia — Luka Petkovic
- Romania — Stefan Lucian Bobu
- Poland — Kacper Krasiński
- Bulgaria — Kaloyan Todorov
- Poland — Oliwia Wika Czarnowska

Project themes.

1. Solar power forecasting
2. Wind power forecasting
3. Load forecasting of a business consumer
4. Load forecasting of a household consumer
5. Energy price forecasting
6. Predicting CO₂ savings from renewables
7. EV energy efficiency prediction

Team work on the
projects.

How to organize your work in 7 steps

1. Do a **research** on the topic
2. Choose **suitable datasets** for YOUR problem
3. Create a Google Colab project and share it with your teammates
4. Work **collectively** on the project together
5. Make a **presentation** with your results - use Google Slides for collaborative work and easier sharing for the final presentation on Thursday
6. Upload your code in **your Github accounts** and share it with us.
7. Add this BIP and the project in your CVs and use it as a future references.

Google Colab and Jupyter Notebooks. *Crash course.*

Before that:

Our suggestion is to use Google Colab, but if you have other experience or prefer something else, you are free to use it, as long as your teammates are OK with it.

In 30 seconds or more

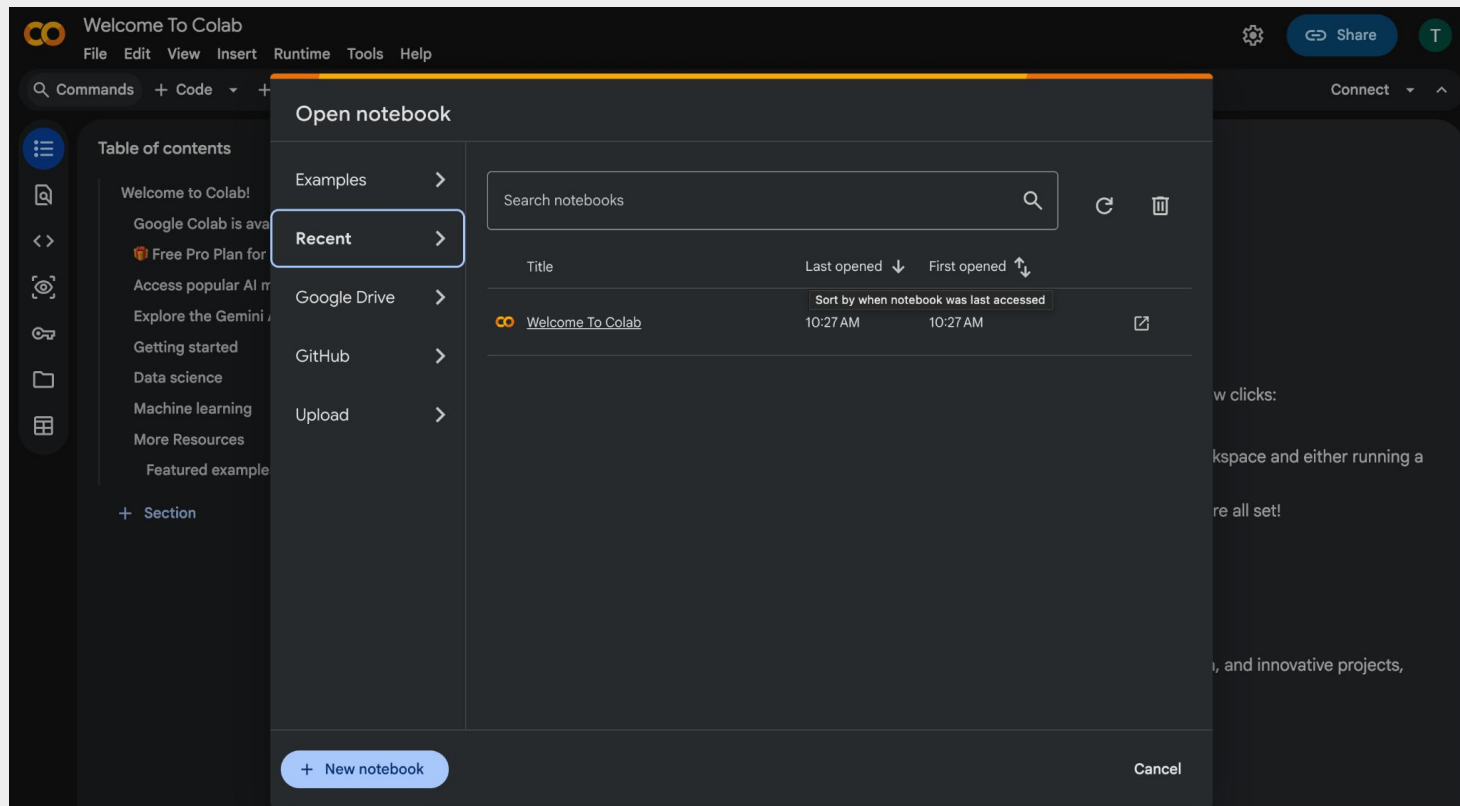
Jupyter Notebook

- Runs locally on your machine.
- Lets you mix:
 - Code (Python, etc.)
 - Text (Markdown)
 - Outputs (plots, tables)
- Great for:
 - Data science
 - ML experiments
 - Teaching

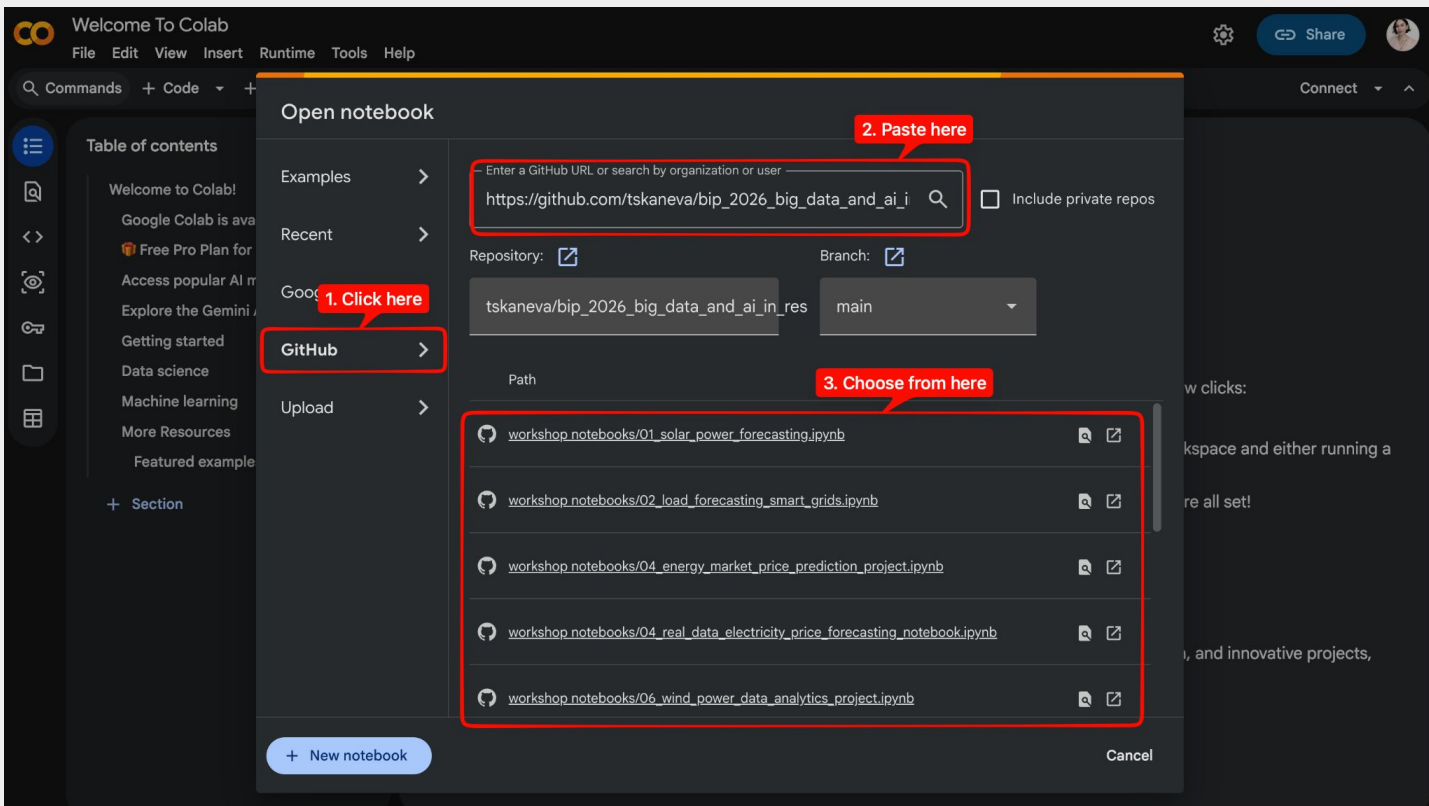
Google Colab

- Runs in the cloud (browser only)
- No setup required
- Free GPU/TPU support
- Integrated with Google Drive
- and can run Jupyter Notebooks (!)

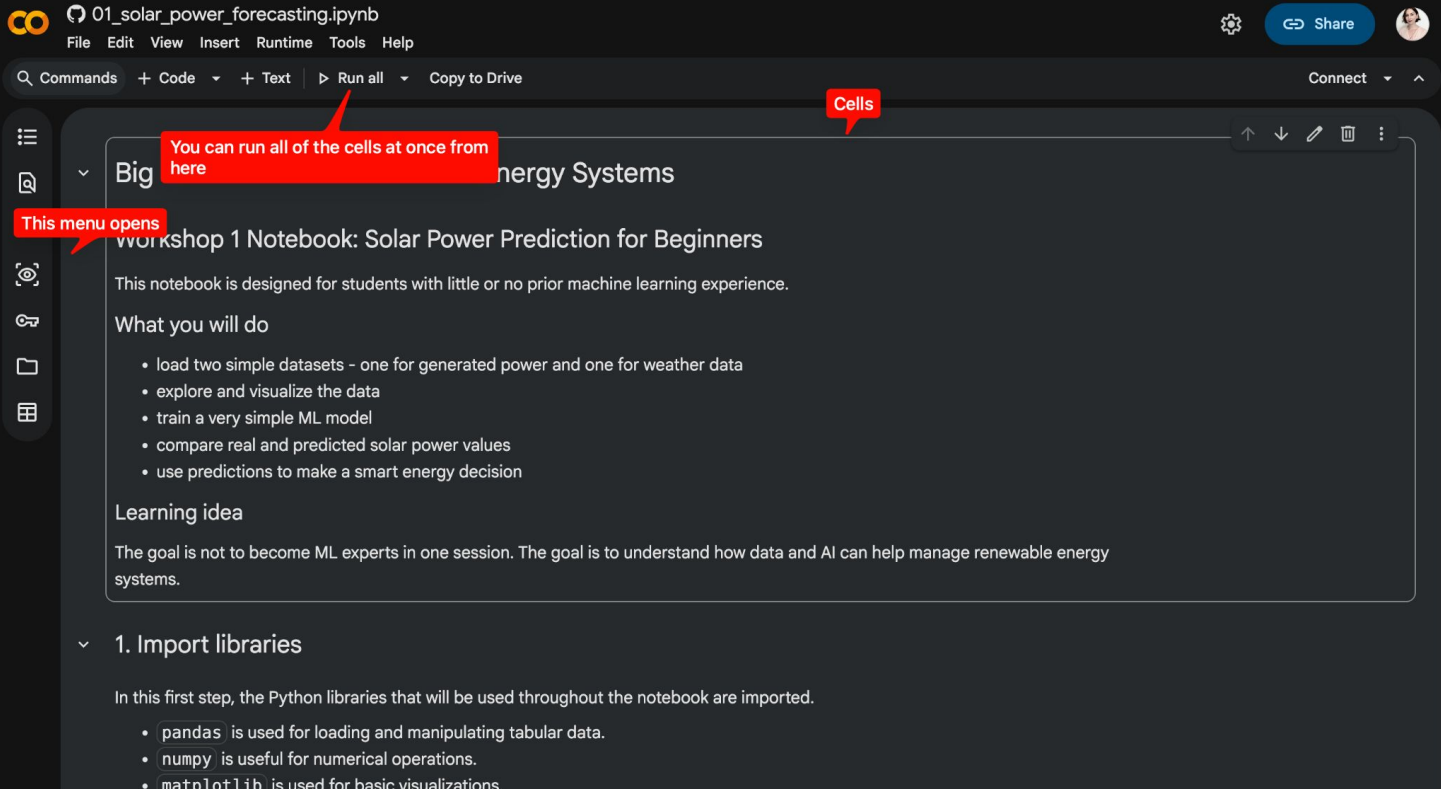
What you see the first time in Google Colab



Opening a Notebook from Github in 3 steps



Opening a Notebook from Github



The screenshot displays a Jupyter Notebook interface for a file named `01_solar_power_forecasting.ipynb`. The top menu bar includes `File`, `Edit`, `View`, `Insert`, `Runtime`, `Tools`, and `Help`. Below the menu, a toolbar contains `Commands`, `+ Code`, `+ Text`, `Run all`, and `Copy to Drive`. On the right side of the toolbar, there is a `Share` button and a user profile icon. A red callout bubble points to the `Run all` button with the text: "You can run all of the cells at once from here".

On the left side, a sidebar menu is visible with icons for `File Explorer`, `Search`, `Recent`, `Help`, `Python Shell`, `Terminal`, and `Output`. A red callout bubble points to the `File Explorer` icon with the text: "This menu opens".

The main content area shows the notebook's title, `Big Data Energy Systems`, and a subtitle, `Workshop 1 Notebook: Solar Power Prediction for Beginners`. The notebook content includes a description: "This notebook is designed for students with little or no prior machine learning experience." and a section titled "What you will do" with a list of tasks:

- load two simple datasets - one for generated power and one for weather data
- explore and visualize the data
- train a very simple ML model
- compare real and predicted solar power values
- use predictions to make a smart energy decision

Below this is a section titled "Learning idea" with the text: "The goal is not to become ML experts in one session. The goal is to understand how data and AI can help manage renewable energy systems."

The notebook is currently on the first cell, titled "1. Import libraries". The text in this cell reads: "In this first step, the Python libraries that will be used throughout the notebook are imported." followed by a list of libraries:

- `pandas` is used for loading and manipulating tabular data.
- `numpy` is useful for numerical operations.
- `matplotlib` is used for basic visualizations.

A red callout bubble points to the `Cells` menu item in the top right corner of the notebook area with the text: "Cells".

**More instructions in the README file in
the Github repo.**

Few important notes

- Variables persist between cells.
- Restarting clears the memory.
- The order of the cells and the execution of the cells matters.
- The path of the data should be declared properly.
- In Colab, there are GPUs, that could be used.
- Colab has a lot of pre-installed packages, but there is an option for installation of new ones.
- After every change of the code, the cell should be rerun.
- Data could be lost after runtime reset in Google Colab.
- Google Drive can be connected (mounted) to the notebook in Colab and the data could be used from the Drive.
- Data could be also save to the Drive.
- Google Colab autosaves the notebooks.
- A handy way of debugging is the usage of the `print()` method.

Datasets.

Some datasets for your projects

- Electricity Load Forecasting -
<https://www.kaggle.com/datasets/saurabhshahane/electricity-load-forecasting>
(Panama case-study)
- Hourly Electricity Consumption and Production (Romania) -
<https://www.kaggle.com/datasets/stefancomanita/hourly-electricity-consumption-and-production>
- Smart Home Dataset with weather Information -
<https://www.kaggle.com/datasets/taranvee/smart-home-dataset-with-weather-information>
- Household Electricity Consumption (Pakistan) -
<https://www.kaggle.com/datasets/alhamdulliah123/pakistan-household-electricity-consumption>
- EV Energy Efficiency Dataset -
<https://www.kaggle.com/datasets/ahmeduzaki/ev-energy-efficiency-dataset>

Some datasets for your projects

- Household Electricity Consumption (240k records) - <https://www.kaggle.com/datasets/thedevastator/240000-household-electricity-consumption-records>
- Building Energy Efficiency Ratings (NYC) - <https://www.kaggle.com/datasets/alistairking/nyc-building-energy-efficiency-ratings/data>
- CO2 and Greenhouse Gas Emissions - <https://www.kaggle.com/datasets/danielrpdias/co2-and-greenhouse-gas-emissions/data>
- European Power Markets Dataset (2015–2024) - <https://www.kaggle.com/datasets/sergionefedov/big-energy-dataset-gas-solarwind-generation>
- Collection of Time Series Dataset (!!! - multiple countries) - <https://www.kaggle.com/datasets/fancifulcrow/collection-of-time-series>

Some datasets for your projects

- Wind Speed vs Spanish Power Prices -
<https://www.kaggle.com/datasets/pythonafroz/wind-speed-vs-spanish-power-prices>
- Wind Power Generation on Electricity Prices -
<https://www.kaggle.com/datasets/saurabhshahane/wind-power-generation-on-electricity-prices>
- European Price Data -
<https://ember-energy.org/data/european-wholesale-electricity-price-data/>
- And many others...

Projects presentations. *Requirements.*

Few notes on the presentations

1. Make a Google Slides presentation (or use any other online tool for shared presentations)
2. In the presentation include:
 - a. Brief description of the problem
 - b. Explanation of the research you did
 - c. Few words about your dataset/s - where did you find them, what data they have, etc.
 - d. Show your solution and explain your logic behind it
 - e. Present your results
 - f. Any other information you think is important
3. Send the URL for the presentation to tskaneva@uni-ruse.bg or ivalova@gmail.com
4. You have max 10 minutes for presentation

Few notes on the workshops AND the presentations

and a *small* change to the program

Wednesday

11:00 - 12:30 - Workshop

12:30 - 14:00 - Lunch Break

14:00 - 14:45 - Laboratory visit

15:00 - 16:00 - Workshop

Thursday

09:00 - 10:30 - Workshop

10:30 - 11:00 - Coffee break

11:00 - 12:30 - Workshop

12:30 - 14:00 - Lunch break

14:00 - 15:30 - Workshop

15:30 - 16:00 - Coffee break

16:00 - 18:00 - Projects Presentation

Thank you! Questions?

You can find this presentation in the Github repo.

Solar power forecasting

Wind power forecasting

Load forecasting of a business consumer

Load forecasting of a household consumer

01

-  – Ana Curic
-  – Teodor-Stefan Zaharia
-  – Piotr Perkowski
-  – Harsh Doshi
-  – Axel Romani

02

-  – Ivona Rados
-  – Dragos-Constantin Toma
-  – Maciej Wozniak
-  – Maria Matyukhina
-  – Kacper Korol

03

-  – Dominik Benak
-  – Stelian Alexandru
-  – Anna Nalewajko
-  – David Hock
-  – Hanna Barylak

04

-  – Mihael Mežnarić
-  – Robert Stefan Cotet
-  – Jan Chalupka
-  – Lien Bradley
-  – Julia Bieganska

Energy price forecasting

Predicting CO2 savings from renewables

EV energy efficiency prediction

05

-  – Nikolina Pejic
-  – Alexandru-Nicolae Badinga
-  – Julia Ordecka
-  – Simona Dragieva
-  – Flavien Norie

06

-  – Filip Bicanic
-  – Danu Cislaru
-  – Dominika Filipiak
-  – Emilia Velikova
-  – Kacper Grobelny

07

-  – Luka Petkovic
-  – Stefan Lucian Bobu
-  – Kacper Krasiński
-  – Kaloyan Todorov
-  – Oliwia Wika Czarnowska